unex

Centro Universitário de Excelência
Sistemas de Informação

MecaniQA: 3ª Entrega
— Otimização e Sintonia Fina —

Autores: Denilson Santos da Silva
Gabriel Arlisson de Souza Santos Torres
João Arthur Nascimento Mascarenhas
Rodrigo Santos da Silva
Tony Cleriston Oliveira dos Santos Junior
Vinicius Cerqueira Oliveira

Feira de Santana, 2026

Agenda

O objetivo é evidenciar o contexto inicial, antes do início da implementação, e apresentar os feitos de cada etapa (por meio dos encontros semanais).

Recapitulação:
Contexto inicial da implementação. Resumo da etapa anterior.

(30/09) Introdução a hiperparâmetros:
Implementação e avaliação dos modelos Random Forest e Gradient Boosting com parâmetros padrão.

(07/10) Otimização de hiperparâmetros:
Otimização de hiperparâmetros com validação temporal para reduzir os erros de previsão.

(14/10) Otimização com Random Search:
Comparação entre Grid Search e Random Search em tempo de execução e precisão.

(21/10) Avaliação final e comparativos:
Comparação dos modelos e baselines por métricas de erro e gráfico de previsões.

Recapitulando...

- Tratamento aplicado: preenchimento dos dias faltantes, interpolação de valores nulos e corte de outliers (limite = média + 3 desvios padrão)
- Features criadas: lags (lag_1, lag_7, lag_30) e médias móveis (7 e 30 dias), sempre defasadas com shift para não vazar dado futuro
- Baselines estabelecidas com validação temporal (TimeSeriesSplit): Naive (MAE 6,48 / RMSE 8,90) e Média Móvel 7 dias (MAE 7,35 / RMSE 8,11 — menor RMSE geral)

Objetivo atual

Sair das baselines, treinar modelos de regressão robustos e otimizar hiperparâmetros.

3ª Entrega: Otimização e Sintonia Fina

(30/09) Introdução a hiperparâmetros

Aplicação do Random Forest e Gradient Boosting com hiperparâmetros padrão, usando lag_1, lag_7, lag_30 e a média móvel de 7 dias, com split 80/20 sem embaralhar. O Random Forest venceu (MAE 3,12, RMSE 3,85, MAPE 13,58%) e seguiu para a otimização.

```
modelos = {
    'Random Forest': RandomForestRegressor(random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(random_state=42),
}

previsoes_ml = {}
for nome, est in modelos.items():
    pipe = montar_pipeline(est)
    pipe.fit(X_treino, y_treino)
    previsoes_ml[nome] = pipe.predict(X_teste)
```

	MAE	RMSE	MAPE (%)
Baseline Naive	6.48	8.90	26.81
Baseline Naive Sazonal 30	10.53	12.59	42.29
Baseline Média Móvel 7	7.35	8.11	30.89
Baseline Média Móvel 30	7.26	8.13	30.38
Random Forest (padrão)	3.12	3.85	13.58
Gradient Boosting (padrão)	3.37	4.18	14.65

(07/10) Otimização de hiperparâmetros

Foi definida uma grade de 36 combinações para o Random Forest (n_estimators, max_depth, min_samples_leaf), otimizada com GridSearchCV + TimeSeriesSplit.

```
%%time
from sklearn.model_selection import GridSearchCV
import time

t0 = time.time()
grid = GridSearchCV(estimator=montar_pipeline(modelos[melhor_nome]
      param_grid=param_grid,
      cv=tcsv,
      scoring='neg_mean_squared_error',
      n_jobs=-1)

grid.fit(X_treino, y_treino)
tempo_grid=time.time() - t0
```

```
CPU times: user 615 ms, sys: 52.6 ms, total: 667 ms
Wall time: 34.1 s
```

Top 5 combinações do Grid Search:

	param_modelo__n_estimators	param_modelo__max_depth	param_modelo__min_samples_leaf	RMSE_cv
24	100	8	5	4.896
33	100	12	5	4.896
26	300	8	5	4.905
35	300	12	5	4.906
25	200	8	5	4.912

```
Modelo: Random Forest
MAE: 3.12 -> 3.03  (redução de 3.0%)
RMSE: 3.85 -> 3.69  (redução de 4.2%)
MAPE (%): 13.58 -> 13.22  (redução de 2.7%)
```

(14/10) Otimização com Random Search

```
%%time
from sklearn.model_selection import RandomizedSearchCV

t0 = time.time()
rand = RandomizedSearchCV(
    estimator=montar_pipeline(modelos[melhor_nome]),
    param_distributions=param_grid,
    n_iter=10,
    cv=tscv,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    random_state=42
)
rand.fit(X_treino, y_treino)
tempo_random=time.time() - t0
```

```
CPU times: user 713 ms, sys: 12.5 ms, total: 726 ms
Wall time: 10.9 s
```

	Tempo em segundos	MAE	RMSE	MAPE (%)
Grid Search	34.15	3.03	3.69	13.22
Random Search	10.91	3.01	3.68	13.20

```
Random Search foi 68% mais rápido
```

O Random Search foi testado como alternativa mais rápida ao Grid Search, sorteando um subconjunto das combinações.

- RandomizedSearchCV com n_iter=10 (10 de 36 combinações), mesma validação (TimeSeriesSplit)
- Melhor combinação: n_estimators=300, min_samples_leaf=5, max_depth=8
- 9,52s contra 36,41s do Grid Search — 74% mais rápido
- Métricas equivalentes (até melhores): MAE 3,01, RMSE 3,68, MAPE 13,20%

(21/10) Avaliação final & comparativos

	MAE	RMSE	MAPE (%)
Baseline Antigo (Baseline Média Móvel 7)	7.35	8.11	30.89
Modelo ML Inicial	3.12	3.85	13.58
Modelo ML Otimizado	3.01	3.68	13.20

```
MAE: 7.35 -> 3.01  (59.0% menos erro)
RMSE: 8.11 -> 3.68  (54.6% menos erro)
MAPE (%): 30.89 -> 13.2  (57.3% menos erro)
```

Considerações finais

- Cenário inicial: baselines ingênuas, MAPE acima de 26%
- Cenário atual: Random Forest + Random Search, MAPE de 13,20%
- Impacto: previsão mais confiável de trocas de óleo apoia planejamento de estoque e escala de equipe
- Aprendizado: Random Search entrega resultado equivalente ao Grid Search em fração do tempo
- Próximos passos: incluir Manutencao_Motor e sazonalidade, testar outros algoritmos, validação mais robusta.

Referências

- SCIKIT-LEARN. **Tuning the hyper-parameters of an estimator (GridSearchCV e RandomizedSearchCV)**. Disponível em: https://scikit-learn.org/stable/modules/grid_search.html. Acesso em: 26 out. 2026.
- BREIMAN, L. Random Forests. **Machine Learning**, v. 45, n. 1, p. 5-32, 2001. DOI: 10.1023/A:1010933404324.
- FRIEDMAN, J. H. Greedy Function Approximation: A Gradient Boosting Machine. **The Annals of Statistics**, v. 29, n. 5, p. 1189-1232, 2001. DOI: 10.1214/aos/1013203451.
- BERGSTRA, J.; BENGIO, Y. Random Search for Hyper-Parameter Optimization. **Journal of Machine Learning Research**, v. 13, n. 10, p. 281-305, 2012.
- HYNDMAN, R. J.; ATHANASOPOULOS, G. Forecasting: Principles and Practice. 3. ed. Melbourne: OTexts, 2021. Disponível em: https://otexts.com/fpp3/. Acesso em: 26 out. 2026.